

Problem Title

3Sum – Find All Unique Triplets That Sum to Zero

Difficulty Level: Medium

Problem Description

You are given an integer array `nums`.

Your task is to find **all unique triplets** `[nums[i], nums[j], nums[k]]` such that:

- $i \neq j, i \neq k, \text{ and } j \neq k$
- $\text{nums}[i] + \text{nums}[j] + \text{nums}[k] == 0$

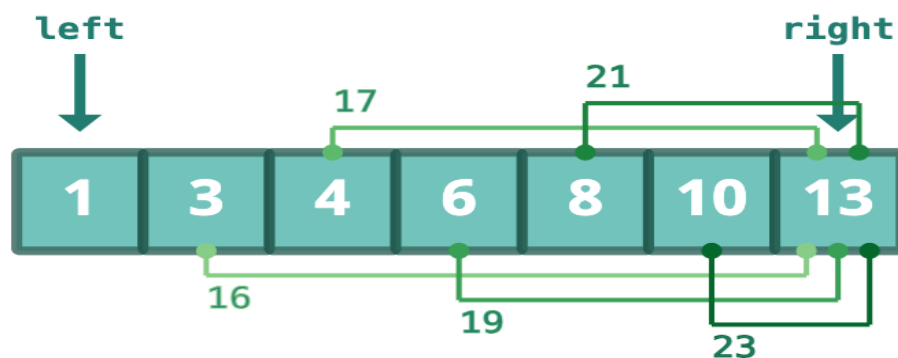
Important:

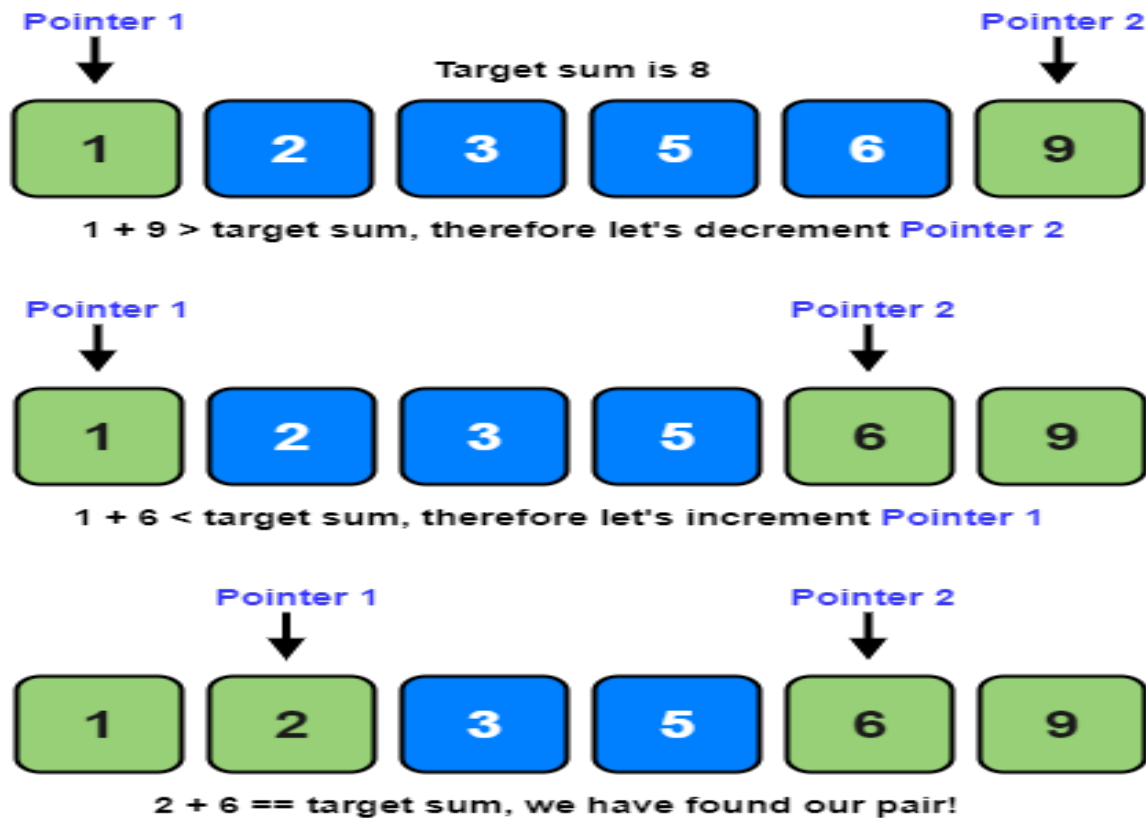
- The solution must **not contain duplicate triplets**.
- The **order of triplets does not matter**.

This problem is commonly used to test:

- Array manipulation
- Sorting + Two Pointers
- Duplicate handling
- Optimization thinking

Visualized Example (Explanation)





3 Sum Problem

Input:

0	1	2	3	4	5
-1	0	1	2	-1	-4

Output: $a+b+c=0$

$$\begin{array}{ccccc} 0 & & 4 & & 3 \\ -1 & + & -1 & + & 2 & = & 0 \\ 0 & & 1 & & 2 \\ -1 & + & 0 & + & 1 & = & 0 \end{array}$$

Example 1

Input: `nums = [-1, 0, 1, 2, -1, -4]`

Step 1: Sort the array

Sorted `nums = [-4, -1, -1, 0, 1, 2]`

Step 2: Fix one element and use two pointers

- Fix -1, search for two numbers with sum +1
- Valid triplets found:
 - `[-1, -1, 2]`
 - `[-1, 0, 1]`

Output:

`[[-1, -1, 2], [-1, 0, 1]]`

Example 2

Input: `nums = [0, 1, 1]`

Output: `[]`

No combination sums to 0.

Example 3

Input: `nums = [0, 0, 0]`

Output: `[[0, 0, 0]]`

Only one valid unique triplet exists.

Input Format

- An integer array `nums` of length `n`
- `n` Space Separated elements

Output Format

- Return a **list of lists**
- Each inner list contains **three integers**
- No duplicate triplets allowed

Constraints

```
3 ≤ nums.length ≤ 3000  
-105 ≤ nums[i] ≤ 105
```

Topic Covered

- Arrays
- Sorting
- Two Pointers
- Hashing (conceptual)
- Duplicate Handling
- Time & Space Complexity

Approaches

1. Brute Force Approach

Idea:

Check all possible triplets using three nested loops.

Algorithm:

1. Use three loops (i, j, k)
2. Check if `nums[i] + nums[j] + nums[k] == 0`
3. Store unique triplets only

Time Complexity: $O(n^3)$

Space Complexity: $O(1)$ (excluding output)

Not suitable for large inputs

Useful only for understanding

2. Optimized Approach (Sorting + Two Pointers)

Idea:

- Sort the array
- Fix one element
- Use two pointers to find the remaining two numbers
- Skip duplicates to avoid repeated triplets

Algorithm:

1. Sort the array
2. Loop i from 0 to $n-3$
3. Use $left = i+1, right = n-1$
4. Move pointers based on sum
5. Skip duplicate values

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$ (excluding output)

Interview-preferred solution

Java Solution (Preferred Language)

```
import java.util.*;

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(nums);

        for (int i = 0; i < nums.length - 2; i++) {

            // Skip duplicate elements
            if (i > 0 && nums[i] == nums[i - 1]) continue;

            int left = i + 1;
            int right = nums.length - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left],
nums[right]));

                    // Skip duplicates
                    while (left < right && nums[left] == nums[left + 1])
left++;
                    while (left < right && nums[right] == nums[right - 1])
right--;

                    left++;
                    right--;
                }
                else if (sum < 0) {
                    left++;
                }
                else {
                    right--;
                }
            }
        }
    }
}
```

```
    }  
    return result;  
}  
}
```

Practice Link (Live)

- **LeetCode**
<https://leetcode.com/problems/3sum/>
- **GeeksforGeeks**
<https://www.geeksforgeeks.org/3sum-problem/>

Video Solution Link

- **YouTube**
<https://www.youtube.com/watch?v=jzZsG8n2R9A>

Learning Outcome

After solving this problem, the learner will be able to:

- Apply sorting + two-pointer technique
- Handle duplicates efficiently
- Reduce brute force complexity
- Solve medium-level array interview problems
- Write clean, optimized Java code